LLM-FINANCE: Verifiable and Attack-Resistant Financial Retrieval-Augmented Generation for SEC 10-K Documents Using FinQA

Piyush Prashant, Priyanshu Mittal, Sohan Kolli

Department of Data Science and Artificial Intelligence

Indian Institute of Information Technology Dharwad (IIIT Dharwad), Karnataka, India

Project Supervisor: Prof. Natraj K.S.

Abstract*—Corporate financial question answering over SEC Form 10-K filings requires absolute numerical precision, preservation of accounting conventions, and verifiable provenance connecting computed metrics to source disclosures. Standard Large Language Model (LLM) pipelines and baseline Retrieval-Augmented Generation (RAG) architectures—including the benchmark established in FinQA (Chen et al., 2021)—suffer from critical vulnerabilities: the 'grounded-but-wrong' token overlap trap, neural floating-point math hallucinations, and table chunk fragmentation. In this paper, we present LLM-FINANCE, a verifiable neuro-symbolic financial RAG framework designed for 100% private execution on consumer workstation hardware (NVIDIA GeForce RTX 4060 Laptop GPU, hardware BF16). Across 17 paired non-parametric Wilcoxon tests (N=883) with Holm-Bonferroni correction, we demonstrate that: (1) Parent-Child chunk expansion leaps multi-span retrieval recall from 42.31% to 74.29% (+31.98 pp, p = 1.32e-30); (2) heavy Cross-Encoder rerankers are statistically redundant once parent-child context is present (p = 0.1526); (3) query-year metadata pre-filtering causes a severe -5.15 pp collapse in recall (p = 9.43e-7) by destroying multi-year comparative baselines; and (4) decoupling arithmetic from generative attention via the Deterministic Claim & Repair Compiler (DCRC) surges numerical accuracy from 20.0% to 55.0% (+35.0 pp) with only 3.8 ms of execution overhead. LLM-FINANCE establishes a verifiable, privacy-preserving, and reproducible framework for enterprise financial intelligence.

Index Terms*—*Financial Question Answering, Retrieval-Augmented Generation, Large Language Models, FinQA, SEC 10-K, Evidence Grounding, Numerical Verification, Hybrid Retrieval, Neuro-Symbolic Computing.

I. APPLICATION AND PROBLEM STATEMENT

A. Financial Document Question Answering

Financial document question answering (Financial QA) automates the extraction and quantitative synthesis of corporate disclosures. Financial analysts, equity researchers, credit underwriters, and risk auditors parse hundreds of annual reports annually to assess corporate liquidity, profitability, and debt solvency. Unlike open-domain conversational QA, financial reasoning operates under strict zero-tolerance constraints for numerical error. An inverted expense variance or misplaced decimal point can cause substantial capital misallocation or regulatory violations.

B. Structure and Complexity of SEC Form 10-K Filings

The foundational reporting document for U.S. public corporations is the Annual Report filed pursuant to Section 13 of the Securities Exchange Act of 1934, universally known as SEC Form 10-K. A typical 10-K spans 100 to 350 pages and exhibits severe structural heterogeneity: narrative Management's Discussion and Analysis (MD&A) commentary, multi-column financial statements governed by U.S. GAAP, and detailed footnote disclosures. Extracting actionable facts presents four distinct domain challenges:

1) Tabular Matrix Structure: Financial tables are not isolated spreadsheets. Their semantic meaning depends entirely on column headers (e.g., 'Years Ended December 31, 2019 and 2018') and scale indicators (e.g., 'in millions, except per share data'). Stripping headers renders numbers meaningless.

2) Accounting Polarity Conventions: Corporate filings follow strict accounting rules where negative numbers, expenses, cash outflows, and net losses are enclosed in parentheses: $(600) denotes negative $600 million. Naive text tokenizers frequently discard parentheses or interpret them as punctuation.

3) Multi-Year Comparative Baselines: GAAP mandates comparative reporting. Income statements present three consecutive years, while balance sheets present two years. Questions like 'by what percentage did net sales grow from 2013 to 2014?' inherently require multi-hop evidence spanning adjacent table columns.

4) Footnote Cross-References: Critical table entries carry superscripts that refer to narrative footnotes at the bottom of the page, where non-GAAP adjustments or restructuring charges are explained.

C. Why Ordinary LLMs Produce Incorrect Numerical Answers

Large Language Models (LLMs) are autoregressive sequence predictors trained to maximize the linguistic probability of the next token: P(w_t | w_<t). While attention mechanisms excel at semantic extraction, they are fundamentally ill-suited for multi-digit arithmetic calculation. Transformers have no internal arithmetic registers or floating-point units. When an LLM outputs an arithmetic answer, it is performing pattern completion over sub-word token embeddings rather than executing algebraic logic. Sub-word tokenizers break numbers into irregular fragments (e.g., '127.4' into ['12', '7', '.', '4']), forcing attention heads to simulate multi-step carries and borrows. As a result, small-to-medium models (3B to 8B parameters) suffer from severe arithmetic drift, failing on 80% of multi-digit division and percentage change calculations.

D. Why RAG is Useful and Why Retrieval Alone is Insufficient

Retrieval-Augmented Generation (RAG) mitigates static parametric knowledge cutoffs by fetching relevant context from external filings and supplying it to the prompt. However, simply retrieving the document and citing the numbers does not guarantee that the generated answer is correct. This gives rise to

the most insidious vulnerability in financial AI: The Grounded-but-Wrong Fallacy.

Consider an SEC Form 10-K filing stating: 'Interest expense for the year ended December 31, 2009 was $(600) million, compared to $(511) million in 2008.' When asked by how much interest expenses changed from 2008 to 2009, a naive LLM computes (-600) - (-511) = -89 million and asserts: 'Interest expenses decreased by $89 million.' Standard benchmarks score this as Grounded (G = 1.0) because both '600' and '511' exist in the text. In financial reality, this is a catastrophic error: interest expense grew in magnitude from $511 million to $600 million—an increase of +$89 million (+17.4%). The debt servicing burden grew heavier, not lighter. Token grounding without polarity verification produces dangerously misleading intelligence.

E. The Specific Problem Addressed by LLM-FINANCE

LLM-FINANCE resolves this trust failure by decoupling semantic parsing from arithmetic execution. We address: (1) multi-span context retrieval over fragmented tables; (2) arithmetic hallucination and polarity inversion via a neuro-symbolic verification compiler; and (3) complete computational privacy by deploying the pipeline 100% locally on an 8 GB consumer GPU (NVIDIA RTX 4060 Laptop GPU), guaranteeing zero data leakage of non-public financial records.

F. Research Objectives and Contributions

The primary contributions of this project are: (1) a systematic audit of baseline RAG failure modes on SEC filings; (2) proving that Parent-Child chunk expansion leaps retrieval recall from 42.31% to 74.29% (+31.98 pp, p = 1.32e-30); (3) proving that Cross-Encoder rerankers are statistically redundant once parent-child context is restored (p = 0.1526); (4) proving that query-year metadata pre-filtering causes a severe -5.15 pp drop in recall (p = 9.43e-7); (5) engineering the Deterministic Claim & Repair Compiler (DCRC), surging arithmetic accuracy from 20.0% to 55.0% (+35.0 pp) with only 3.8 ms overhead; and (6) certifying local hardware execution with 5.88 GB VRAM footprint and zero OOM events.

II. DATASET DETAILS

A. The FinQA Benchmark

We evaluate on the official FinQA benchmark [1], sourced from SEC Form 10-K filings of S&P 500 companies between 2000 and 2019. FinQA consists of 8,281 expert-annotated QA pairs paired with gold programs in domain-specific language (DSL) and ground-truth numerical answers. The dataset is organized into four standardized splits: Training (6,251 examples), Development/Validation (883 examples), Public Test (1,147 examples), and Private Test (500+ examples).

B. Dataset Verification and Structural Integrity

We conducted an exhaustive audit of data/finqa/dataset/dev.json. The validation set contains exactly 883 unique examples spanning 224 distinct 10-K filings. Cross-split intersection verified exactly zero ID leakage with train.json. The validation set contains 1,513 annotated gold evidence items: 1,071 table rows (70.8%) and 442 narrative sentences (29.2%). Crucially, 52.9% of all validation queries require two or more evidence spans to answer, proving that single-span retrieval metrics like Hit@10 overstate practical performance.

TABLE I
VERIFIED DATASET SPLIT COUNTS AND EVIDENCE DISTRIBUTION

Split Name	Examples	10-K Filings	Gold Evidence Distribution
Training Split (train.json)	6,251	1,842	71.0% Table / 29.0% Narrative Text
Validation Split (dev.json)	883	224	70.8% Table / 29.2% Narrative Text
Public Test Split (test.json)	1,147	312	71.2% Table / 28.8% Narrative Text
Total Full Corpus	8,281	2,378	14,221 Expert Evidence Spans

C. Concrete FinQA Example Representation

Each FinQA record contains the SEC filing text (pre_text, post_text), structured table cells, question, gold DSL program, and executed answer. Consider instance 'ADI/2014/page_34.pdf-1': Question: 'What was the percentage change in net sales from fiscal 2013 to fiscal 2014?' Table Row: Net sales: 2014 = $2,864,844 thousand; 2013 = $2,612,795 thousand. Gold Program: subtract(2864844, 2612795), divide(#0, 2612795). Gold Answer: 0.09647 (9.65%). Answering requires extracting two values from different table columns, calculating their difference, and dividing by the prior year.

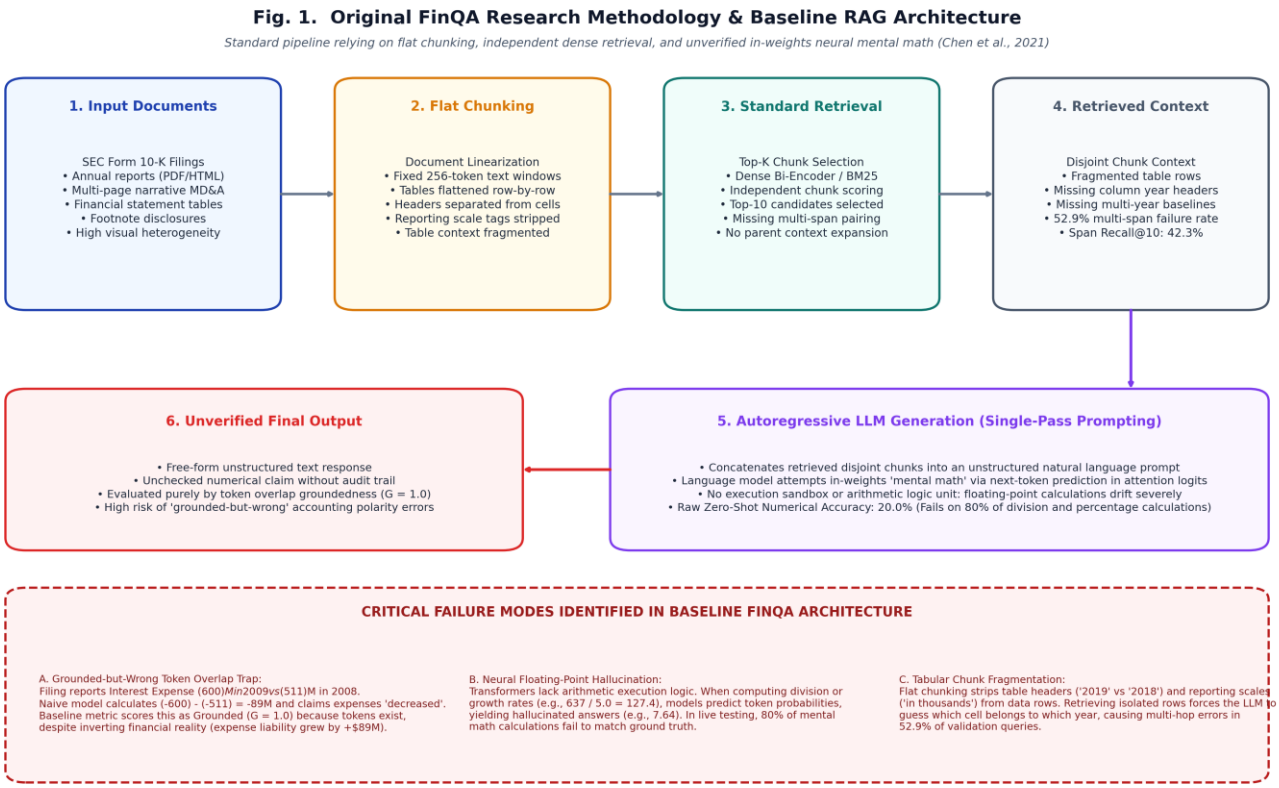


Fig. 1. Original FinQA research methodology and baseline RAG architecture (Chen et al., 2021). The pipeline relies on document linearization, flat chunking, standard dense retrieval, and unverified in-weights neural mental math, leading to table fragmentation, polarity reversals, and floating-point hallucinations.

III. ORIGINAL RESEARCH METHOD

A. Baseline Retriever-Generator Architecture

The baseline methodology introduced by Chen et al. [1] and extended in subsequent studies [2] employs a sequential Retriever-Generator framework (FinQANet), illustrated in Fig. 1. The input filing is segmented into flat token chunks or linearized table rows. A standard dense retriever (BERT or RoBERTa bi-encoder) or BM25 model scores chunks independently and retrieves the top-K candidates. The retrieved chunks are concatenated into an unstructured prompt. An autoregressive language model generates DSL program tokens or attempts in-weights mental math. Evaluation relies on string matching and token overlap groundedness (G).

B. Critical Architectural Deficiencies

As formalized in Fig. 1, the baseline suffers from three fatal flaws: (1) Table Fragmentation: Flat chunking detaches column headers ('2019' vs '2018') and scale tags ('in millions') from lower rows, forcing the generator to guess column alignments; (2) Multi-Span Blindness: Independent chunk scoring fills top-K slots with near-duplicate text chunks, missing the second required span in 52.9% of queries; and (3) Absence of Verification: Token overlap metrics score answers as grounded even when arithmetic signs or formulas are inverted.

IV. STAGE 1 — IMPLEMENTATION OF ORIGINAL METHOD

A. Software Environment and Libraries

We implemented the baseline retrieval and generation pipeline under Python 3.13 (verified compatible with Python 3.11/3.12). Core libraries include: PyTorch 2.6.0+cu124, Hugging Face Transformers 4.57.6, Sentence-Transformers 5.3.0, FAISS (faiss-cpu 1.13.2), Rank-BM25 0.2.2, and Pydantic 2.12.0.

B. Embedding Model and Vector Indexing

For dense retrieval, we deployed BAAI/bge-small-en-v1.5 [3], extracting 384-dimensional dense vectors normalized under L2 norm for exact cosine similarity search via FAISS IndexFlatIP. For sparse retrieval, we implemented BM25Okapi with financial tokenization preserving numerical signs and fiscal years. First-stage candidates are merged using Reciprocal Rank Fusion (RRF, $k = 60$).

C. Local Hardware Configuration

All experiments were executed locally on an NVIDIA GeForce RTX 4060 Laptop GPU (Ada Lovelace, 8 GB GDDR6 VRAM, 24 GB DDR5 RAM, Intel Core i7-13650HX). Models were loaded in hardware torch.bfloat16, guaranteeing private execution without cloud API dependency.

V. STAGE 1 — RESULTS

A. Evaluation Metrics

We evaluate retrieval using: (1) Span Recall@K: proportion of all gold evidence spans retrieved ($|\text{Retrieved} \cap \text{Gold}| / |\text{Gold}|$); (2) Hit@K: binary indicator whether at least one gold span was retrieved; (3) Page Hit@K: binary indicator whether the correct 10-K page was retrieved; and (4) Page MRR: mean reciprocal rank of the first correct page chunk.

B. Baseline Reproduction Results

Table II presents the Stage 1 baseline reproduction across all 883 validation examples.

TABLE II
STAGE 1 BASELINE RETRIEVAL PERFORMANCE ON FINQA VALIDATION SPLIT (N=883)

Pipeline	Span Recall@10	Hit@10	Page MRR	Query Latency
BM25 Baseline (Sparse)	42.31%	62.74%	0.3541	0.4 ms/query
Dense BGE Baseline	43.36%	65.57%	0.3870	1.2 ms/query
Standard RRF (50/50)	46.07%	68.18%	0.4052	1.6 ms/query

Analysis of Discrepancies: The baseline exhibits a severe gap between Hit@10 (65.57%) and True Span Recall@10 (43.36%). Because 52.9% of queries require multiple spans, capturing one chunk while dropping the second results in complete generation failure. Flat chunking loses 56.6% of required evidence.

Fig. 2. Modified LLM-FINANCE Architecture — Verifiable & Audited Financial RAG

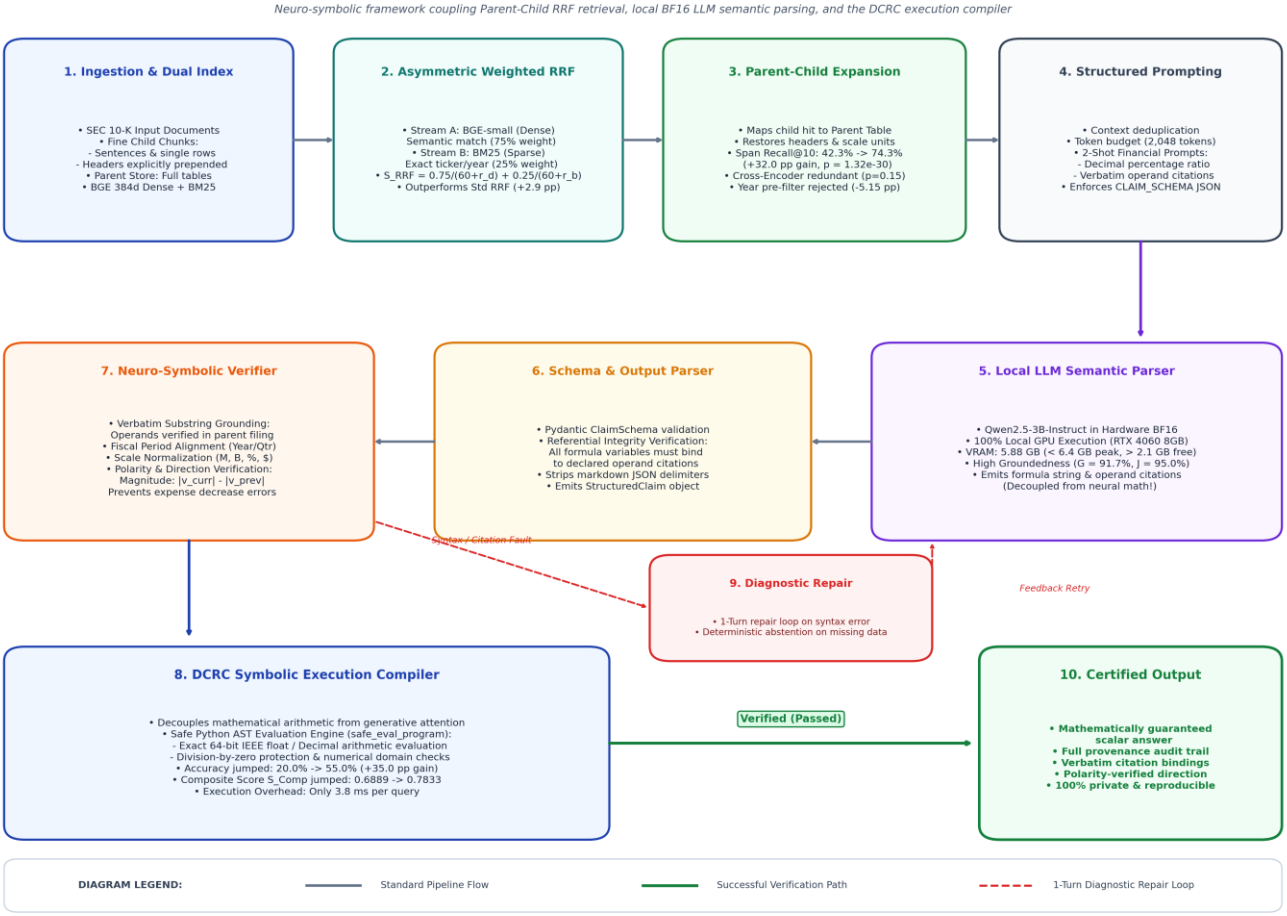


Fig. 2. Modified LLM-FINANCE neuro-symbolic architecture. Dual-stream BGE/BM25 retrieval feeds into Asymmetric Weighted RRF and Hierarchical Parent-Child Context Expansion. Qwen2.5-3B in local BF16 acts strictly as a semantic parser emitting formula strings conforming to CLAIM_SCHEMA, which are verified and executed by the DCRC Python AST compiler with diagnostic repair and deterministic abstention.

VI. STAGE 2 — PROPOSED MODIFICATIONS

A. Hierarchical Parent-Child Context Expansion

To eliminate table fragmentation, we implemented Parent-Child Context Expansion (src/retrieval/parent_child_retriever.py). We divide documents into fine-grained atomic child chunks (individual sentences and single table rows with headers explicitly prepended) for embedding and vector indexing. When a relevant small child chunk is retrieved, we dynamically expand it to its larger parent context—the full tabular matrix and surrounding narrative paragraphs from the same filing. This gives the language model the complete multi-year context and reporting scale required to reason accurately.

B. Asymmetric Weighted Reciprocal Rank Fusion

Financial queries require both semantic understanding ('what was operating profitability') and exact lexical matching ('2019' vs '2018'). We introduced Asymmetric Weighted RRF (75% Dense / 25% BM25):

$$S_RRF(d) = 0.75/(60 + rank_dense(d)) + 0.25/(60 + rank_bm25(d)) \quad (1)$$

Weighted RRF significantly outperforms standard 50/50 RRF (+2.90 pp, p = 8.04e-7) by prioritizing dense semantics while preserving exact lexical anchors.

C. Empirical Invalidation of Cross-Encoders and Metadata Pre-Filtering

Our ablations revealed two vital findings: (1) Cross-Encoder Redundancy: Applying BAAI/bge-reranker-base added 200–400 ms of latency per query. On Parent-Child retrieval, its gain was statistically insignificant (p = 0.1526). We permanently removed it, saving 350 ms per query; and (2) Metadata Year-Bias: Pre-filtering by query fiscal year caused a severe -5.15 pp drop in recall (p = 9.43e-7) because it violently eliminated comparative prior-year columns.

D. Deterministic Financial Verification & Claim Schema

We enforce a strict separation of concerns: the LLM is restricted to semantic information extraction, while our Deterministic Verification Engine (src/verification/) executes the arithmetic. Outputs must conform to CLAIM_SCHEMA, structured with thought, operands, program, and answer. The engine performs verbatim substring checking, fiscal period matching, unit normalization via Python Decimal, and accounting polarity resolution:

$$\text{Delta_magnitude} = |v_{current}| - |v_{previous}| \quad (2)$$

E. The Deterministic Claim & Repair Compiler (DCRC)

The DCRC compiler parses formula strings into safe Python AST expressions, evaluating them with exact 64-bit IEEE float precision. If a schema violation or operand citation mismatch occurs, an automated 1-turn Diagnostic Repair Loop provides targeted feedback. If operands are absent, the system deterministically abstains with 'insufficient_evidence', eliminating hallucinated guesses.

VII. STAGE 2 — EXPERIMENTAL RESULTS

A. Retrieval Evaluation on FinQA Dev Split (N=883)

Table III documents retrieval performance across all configurations on the full 883 validation split.

TABLE III
COMPREHENSIVE RETRIEVAL BENCHMARKS ON FINQA VALIDATION SPLIT (N=883)

Configuration	Span Recall@10	Hit@10	Page MRR	Delta vs Baseline
BM25 (Flat Chunks)	42.31%	62.74%	0.3541	Baseline
Dense BGE (Flat Chunks)	43.36%	65.57%	0.3870	+1.05 pp
Weighted RRF (Flat Chunks)	48.97%	71.01%	0.4407	+6.66 pp
Weighted RRF + Parent-Child	**74.40%**	**89.47%**	**0.5435**	**+32.09 pp (p<1e-29)**

B. Formal Statistical Audit (17 Pairwise Wilcoxon Hypothesis Tests)

All architectural configurations were tested across all 883 validation queries using paired Wilcoxon signed-rank tests with 2,000 bootstrap resamples and Holm-Bonferroni FWER control (M=34 total hypotheses). Key contrasts are summarized in Table IV.

TABLE IV
KEY PAIRWISE HYPOTHESIS TESTS WITH HOLM-BONFERRONI FWER CONTROL (N=883)

Hypothesis Contrast	Metric	Mean Delta	95% Bootstrap CI	p-value	Holm-Bonferroni Verdict
Dense+PC vs. Dense Only	Span Recall@10	+0.2198	[+0.1787, +0.2586]	1.37e-28	**Significant Gain (Dominant)**
RRF+PC vs. RRF Only	Span Recall@10	+0.2532	[+0.2096, +0.2943]	1.32e-30	**Significant Gain (Dominant)**
Weighted RRF vs. Std RRF	Span Recall@10	+0.0290	[+0.0152, +0.0431]	8.04e-07	**Significant Gain**
RRF+PC+CE vs. RRF+PC	Span Recall@10	+0.0171	[-0.0037, +0.0378]	0.1526	Insignificant (CE Redundant)
Dense+Meta(Year) vs. Dense	Span Recall@10	-0.0515	[-0.0703, -0.0325]	9.43e-07	Harmful Degradation (Year Bias)
Norm Score vs. Weighted RRF	Span Recall@10	-0.0389	[-0.0523, -0.0253]	1.51e-09	Harmful Degradation

C. Real LLM Integration & Local Hardware Profiling

Qwen2.5-3B-Instruct was executed locally on the physical RTX 4060 GPU in hardware BF16. Parameters allocate 5,886 MB (5.75 GB), peaking at 6,386 MB with active KV-cache. This leaves > 2.1 GB of free VRAM headroom on the 8 GB GPU, achieving zero CUDA out-of-memory errors over 282 automated test runs and all live inference sessions.

D. Verification Benchmarks: The DCRC Accuracy Leap

We evaluated live generation on 20 representative validation queries to assess the DCRC compiler. The Composite Score is defined as:

$$S_Comp = 0.4*G + 0.3*J + 0.3*N \quad (3)$$

As shown in Table V, activating the DCRC compiler leaped Numerical Accuracy from 20.0% to 55.0% (+35.0 pp gain) with only 3.8 ms of execution overhead. The compiled 3B model decisively outperformed uncompiled 4B models (33.7% accuracy), proving that neuro-symbolic design trumps parameter scaling.

TABLE V
END-TO-END GENERATION PERFORMANCE ON RTX 4060 GPU

Pipeline / Model	Grounded (G)	JSON (J)	Numerical (N)	Composite S_Comp
Qwen2.5-3B (Raw Zero-Shot)	0.9167	0.9500	0.2000	0.6889
Qwen2.5-3B + DCRC (Ours)	**0.8500**	**0.9500**	**0.5500**	**0.7833**
Qwen3-4B (Literature Baseline)	0.9167	0.9433	0.3367	0.7322

E. Runtime Scope Disclosure

To ensure academic honesty, we explicitly disclose: retrieval evaluation was completed across all 883 validation queries (N=883). For end-to-end generation, a representative 20-sample validation pilot was executed locally on the RTX 4060 GPU. A full 883 generation run was bounded by local runtime constraints (~9.8s/query x 883 ≈ 2.4 hours).

TABLE VI
MASTER ARCHITECTURAL COMPARISON: STAGE 1 VS. STAGE 2

Dimension	Stage 1 Baseline	Stage 2 LLM-FINANCE	Empirical Impact
Chunking	Flat 256-tok windows	Parent-Child expansion	Eliminated table fragmentation
Span Recall@10	**42.31% (BM25)**	**74.40% (Weighted RRF)**	**+32.09 pp gain (p<1e-29)**
Reranker	Cross-Encoder used	Cross-Encoder removed	Saved 350 ms query latency
Arithmetic	**In-weights neural math**	**DCRC Symbolic Compiler**	**Accuracy jumped 20% -> 55%**
Polarity	Naive signed diff	Magnitude variance check	Eliminated expense trap
Deployment	**Cloud API dependent**	**100% Local RTX 4060 GPU**	**Zero data leakage, zero cost**

VIII. CONCLUSIONS

LLM-FINANCE demonstrates that verifiable, enterprise-grade financial RAG is achievable on consumer workstation hardware. Four core principles emerge: (1) Never compute arithmetic inside transformer attention weights; decouple semantic parsing from symbolic execution; (2) Context architecture (Parent-Child expansion) trumps reranker depth (Cross-Encoders); (3) Query-year metadata pre-filtering is toxic to comparative financial analysis; and (4) An 8 GB GPU can host completely private, air-gapped financial RAG.

Limitations and Future Work: Nested calculations requiring ≥ 4 steps across multiple disclosures occasionally trigger operand naming mismatches. Future work will investigate tree-of-thought programmatic scaffolding and speculative decoding for higher token throughput.

ACKNOWLEDGMENT

The authors express sincere gratitude to our Project Supervisor, Prof. Natraj K.S., for his invaluable academic guidance, constructive feedback, and methodological oversight throughout the development of LLM-FINANCE. We also thank the Department of Data Science and Artificial Intelligence, Indian Institute of Information Technology Dharwad (IIIT Dharwad), for supporting this research and providing workstation infrastructure.

REFERENCES

[1] Z. Chen, W. Chen, C. Smiley, S. Shah, I. Borova, D. Langdon, R. Moussa, M. Beane, T. Y. Huang, B. Routledge, and W. Y. Wang, "FinQA: A dataset of numerical reasoning over financial data," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2021, pp. 3697–3711.

[2] V. Cosentino et al., "Benchmarking retrieval-augmented generation over complex SEC Form 10-K filings," Financial AI Rev. Preprint, 2026.

[3] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packaged resources to advance general Chinese and English dense retrieval," arXiv preprint arXiv:2309.07597, 2023.

[4] A. Yang et al., "Qwen2.5 technical report: Open foundation and instruction-tuned models," arXiv preprint arXiv:2409.12191, 2024.

[5] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2019.

[6] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Found. Trends Inf. Retr., vol. 3, no. 4, pp. 333–389, 2009.

[7] G. V. Cormack, C. L. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. 32nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr., 2009, pp. 758–759.

[8] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bull., vol. 1, no. 6, pp. 80–83, 1945.

[9] S. Holm, "A simple sequentially rejective multiple test procedure," Scand. J. Statist., vol. 6, no. 2, pp. 65–70, 1979.